Documento de Arquitectura de Software (DAS)

Caso 7 — Alerta temprana de deslizamientos

Curso: Arquitectura de Software — Uniremington
Docente: Gloria Amparo Lora Patiño
Estudiante: Alejandro Acosta
Versión del documento: 0.3 — Fecha: 14/10/2026
Repositorio: alerta-temprana-deslizamientos

0. Enunciado del Caso 7 (texto original)

Contexto del caso

Una red de 8 sensores de humedad instalados en zona de ladera reporta datos por lotes cada hora, vía SMS, a un celular que un técnico revisa manualmente. No hay presupuesto para reemplazar los sensores.

Palabras del cliente

“El deslizamiento de la vereda El Roble ocurrió a las 3 de la madrugada. El reporte del sensor llegó a las 4am, cuando el técnico finalmente lo revisó. Para entonces ya era tarde.”

Datos del caso

Infraestructura fija: 8 sensores existentes, no reemplazables por restricción de presupuesto.

Frecuencia actual: reporte por lotes cada hora, revisión manual.

Atributo de calidad crítico: disponibilidad y tiempo de respuesta (una alerta tardía no sirve).

Restricción técnica dura: no se puede cambiar el hardware de los sensores, solo el procesamiento de los datos que ya envían.

Reto para el arquitecto

Diseñar cómo procesar y alertar sobre los datos que ya emiten los sensores existentes de forma casi inmediata, sin poder modificar el hardware, priorizando disponibilidad y tiempo de respuesta por encima de cualquier otro atributo.

Ficha de Drivers Arquitectónicos — Entregable 1

Caso 7 · Alerta temprana de deslizamientos · Cierre Unidad 1

1. Requisitos funcionales (RF-01 a RF-10)

ID	Requisito funcional	Origen / justificación
RF-01	Recibir y registrar los SMS entrantes de los 8 sensores	Explícito: “reporta datos por lotes cada hora, vía SMS”
RF-02	Interpretar el contenido de cada SMS (sensor, hora, valor de humedad)	Derivado: necesario para procesar sin tocar el hardware
RF-03	Almacenar el histórico de lecturas por sensor	Derivado: se necesita para detectar tendencia, no solo el dato actual
RF-04	Evaluar cada lectura contra un umbral de riesgo configurado	Derivado del reto: “alertar sobre los datos que ya emiten”
RF-05	Detectar tendencia de aumento sostenido en lecturas consecutivas	Derivado de la cita del cliente: la alerta a tiempo exigía ver el patrón antes del lote crítico
RF-06	Generar una alerta cuando se supere el umbral o se detecte tendencia de riesgo	Explícito: “alertar... de forma casi inmediata”
RF-07	Notificar la alerta al responsable por un canal más inmediato que la revisión manual del celular	Derivado: la revisión manual fue la causa raíz del retraso en el caso
RF-08	Permitir configurar/ajustar los umbrales de riesgo	Derivado: sin esto no hay forma de calibrar el sistema al escenario real
RF-09	Registrar historial de alertas generadas (trazabilidad)	Derivado de buenas prácticas de trazabilidad
RF-10	Consultar el estado de cada sensor (última lectura, si reporta con normalidad)	Derivado: mitiga el riesgo de sensor caído o dañado

2. Atributos de calidad — ISO/IEC 25010

AC	Característica	¿Aplica?	Justificación	Prioridad
AC-01	Adecuación funcional	Sí	Debe procesar correctamente las lecturas de los 8 sensores y generar la alerta correspondiente sin omitir eventos de riesgo real	Media
AC-02	Eficiencia de desempeño	Sí	“Alertar... de forma casi inmediata”; el caso mide el fracaso en horas (3am vs 4am)	Alta
AC-03	Compatibilidad	Sí	Debe integrarse con el canal SMS y los sensores ya existentes, sin poder cambiarlos	Media
AC-04	Usabilidad	Sí	El técnico revisa manualmente hoy; el nuevo sistema debe operarlo alguien con conocimiento técnico bajo/medio	Media
AC-05	Confiabilidad (fiabilidad)	Sí	“Atributo de calidad crítico: disponibilidad”; un solo técnico y un solo celular es punto único de falla	Alta
AC-06	Seguridad	Sí	No hay dato explícito, pero una alerta falsa o manipulada sobre riesgo de vida tiene consecuencia grave	Baja
AC-07	Mantenibilidad	Sí	Restricción de hardware fijo: toda evolución (umbrales, lógica de tendencia) debe hacerse solo en software	Alta
AC-08	Portabilidad	No aplica	El caso no menciona necesidad de correr en otra plataforma o migrar de entorno	No aplica

3. Restricciones técnicas

RT	Restricción	Origen
RT-01	No se puede reemplazar ni modificar el hardware de los 8 sensores; solo se interviene el procesamiento posterior	Explícito en el caso
RT-02	El canal es SMS por lotes cada hora; no hay flujo de datos continuo ni streaming	Explícito: “reporta datos por lotes cada hora, vía SMS”
RT-03	El único punto de recepción actual es un celular revisado manualmente; no hay infraestructura de recepción adicional	Derivado del caso
RT-04	Solo se dispone de humedad como variable; no hay lluvia, inclinación u otros sensores	Derivado, limita cualquier modelo predictivo

4. Restricciones administrativas

RA	Restricción	Origen
RA-01	No hay presupuesto para reemplazar los sensores	Explícito en el caso
RA-02	No hay evidencia de presupuesto para infraestructura adicional (servidores, nube, personal técnico)	Derivado, coherente con restricciones como problema de inversión
RA-03	No hay turno de guardia nocturno confirmado; el reporte de las 3am solo se revisó hasta las 4am	Derivado de la cita del cliente

5. Pregunta al cliente

¿Los reportes de las horas previas al deslizamiento (por ejemplo 1am y 2am) ya mostraban un aumento progresivo de humedad, o el salto ocurrió únicamente en el lote de las 3am?

Esta pregunta expone el supuesto que sostiene todo el diseño: si hubo tendencia previa, el sistema necesita detectar tendencia (RF-05) y no solo un umbral fijo; si fue un salto súbito, ni la alerta más rápida habría llegado a tiempo, y el problema real es el intervalo físico de una hora entre lecturas, no el software.

Sección 1: Introducción y Drivers Arquitectónicos

1.1 Visión general del negocio y alcance

Problema de negocio: una red de 8 sensores de humedad instalados en zona de ladera reporta datos por lotes cada hora, vía SMS, a un celular revisado manualmente por un técnico. La revisión tardía impide alertar a tiempo, como ocurrió en el deslizamiento de la vereda El Roble.

Propuesta de solución: el sistema permite al equipo de vigilancia procesar automáticamente los reportes SMS de los sensores existentes para generar y notificar alertas de riesgo de deslizamiento sin depender de la revisión manual de un único técnico.

Alcance (in-scope): recepción e interpretación de SMS de los 8 sensores existentes; almacenamiento histórico de lecturas; evaluación de umbral y detección de tendencia; generación y notificación de alertas por múltiples canales; configuración de umbrales; consulta de estado de sensores e historial de alertas.

Fuera de alcance (out-of-scope): reemplazo o modificación del hardware de los sensores; predicción climática con variables distintas a humedad (lluvia, sismos); coordinación de evacuación o respuesta de emergencia física; aplicación móvil nativa para el ciudadano final; cambio del canal de transmisión de los sensores (sigue siendo SMS).

1.2 Historias de Usuario arquitectónicamente significativas

ID	Historia de usuario	¿Por qué es significativa?
HU-01	Como sistema, quiero generar y registrar una alerta cuando una lectura supere el umbral o muestre tendencia de riesgo, para notificar el peligro con el menor retraso posible.	Ejercita persistencia, lógica de negocio crítica e integración con canal de notificación. Base del CRUD del Corte Vertical (entidad “Alerta”).
HU-02	Como administrador, quiero configurar los umbrales de riesgo por sensor, para ajustar la sensibilidad del sistema sin depender de cambios en el hardware.	Ejercita seguridad/autorización y conecta directo con la restricción técnica de hardware fijo.
HU-03	Como técnico, quiero consultar el estado y última lectura de cada sensor, para detectar si alguno dejó de reportar.	Ejercita confiabilidad y tolerancia a fallos.

1.3 Atributos de calidad prioritarios — escenarios medibles

Atributo	Escenario de calidad	Métrica objetivo	¿Cómo se verificará?
Eficiencia de desempeño	Llega un SMS con una lectura de humedad, en operación normal	El sistema procesa y evalúa la lectura en ≤ 5 segundos desde su recepción	Comparar timestamp de llegada del SMS vs. timestamp de evaluación, en logs
Confiabilidad	Se genera una alerta de riesgo, en cualquier horario incluida la madrugada sin guardia activa	La alerta se entrega por al menos 2 canales/destinatarios independientes en el 100% de los casos	Prueba funcional: desactivar un canal y confirmar que el otro entrega la alerta
Mantenibilidad	Se necesita ajustar el umbral de riesgo o la regla de tendencia	El cambio afecta máximo 1-2 archivos de configuración, sin tocar el módulo de recepción de SMS ni el esquema de BD	Revisión de código del corte vertical

Nota: el tiempo de procesamiento (≤ 5 s) mide la latencia del software, no el intervalo de una hora entre lecturas del sensor, el cual es una restricción de hardware (RT-02) que no puede modificarse.

1.4 Restricciones

Tipo	Restricción
Técnica	No se puede reemplazar ni modificar el hardware de los sensores
Técnica	Canal SMS por lotes cada hora, sin streaming continuo
Operativa	Sin presupuesto para sensores ni infraestructura adicional
Operativa	Sin evidencia de turno de guardia nocturno
De tiempo	Corte vertical funcional Clase 6 (07-nov); DAS v1.0 Clase 7 (14-nov); entrega final Clase 8 (21-nov)
Normativa	No identificada explícitamente en el caso

Sección 2: Estrategia Arquitectónica y Tecnológica

2.1 Patrón arquitectónico principal

Patrón seleccionado: N-Capas (presentación, negocio y datos).

Justificación: Se selecciona el patrón N-Capas (presentación, negocio y datos) porque el atributo prioritario Mantenibilidad (AC-07) exige que ajustar un umbral o la regla de tendencia afecte máximo 1-2 archivos de configuración, sin tocar el módulo de recepción de SMS ni el esquema de la base de datos; la separación en capas concentra las reglas en una sola capa de negocio, aislada de la entrada y de la persistencia. Esa misma capa expone los canales de notificación detrás de una interfaz común con al menos dos implementaciones independientes, lo que sostiene la Confiabilidad (AC-05): si un canal falla, el otro entrega la alerta. Al desplegarse el backend como una sola aplicación, la lectura se procesa dentro del mismo proceso, sin saltos de red internos, lo que favorece la Eficiencia de desempeño (AC-02, ≤ 5 s). Se descarta la Arquitectura Hexagonal porque su estructura adicional contradice la restricción de tiempo (sección 1.4) de un proyecto individual con un solo desarrollador, y se descartan Microservicios y la arquitectura orientada a eventos con broker porque su costo operativo contradice la restricción de presupuesto (RA-02) y añade puntos de falla frente a la Confiabilidad (AC-05).

2.2 Pila tecnológica

Cada tecnología se conecta con un atributo de calidad prioritario (sección 1.3) o con una restricción (sección 1.4), y se indica la alternativa descartada.

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación
Lenguaje backend	Java	21 LTS	Python	Restricción de tiempo (sección 1.4): lenguaje de dominio previo del estudiante (dos asignaturas cursadas con Spring Boot) en un proyecto individual con plazo corto; Python implicaría una curva de aprendizaje nueva.
Framework	Spring Boot	4.1.x	FastAPI, Django	AC-07 (Mantenibilidad): su estructura Controller-Service-Repository materializa las capas del patrón N-Capas y aísla las reglas de negocio en la capa de servicio; incluye validación de datos y documentación OpenAPI. Restricción de tiempo: dominio previo del estudiante.
Acceso a datos	Spring Data JPA (Hibernate) + Flyway	Según Spring Boot	JDBC / SQL nativo	AC-07: mantiene el modelo de datos separado de las reglas de negocio mediante repositorios y versiona los cambios del esquema con migraciones.

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación
Base de datos	PostgreSQL	18	MongoDB	Datos relacionales (sensor, lectura, alerta) que requieren integridad referencial y restricciones (FK, CHECK, UNIQUE) para el historial de RF-03 y RF-05; licencia libre (RA-02).
Frontend	React (Vite) como aplicación separada	React 19.x	HTML y JavaScript servidos desde Spring Boot	AC-04 (Usabilidad): interfaz mínima para consultar alertas y estado de sensores (HU-03) y configurar umbrales (HU-02). Restricción de tiempo: dominio previo del estudiante en React; consume la API REST del backend con JWT.
Autenticación	Spring Security + JWT, con BCrypt y roles administrador y técnico	Según Spring Boot	Sesiones en servidor	AC-06 (Seguridad) y HU-02: solo el administrador modifica umbrales, porque un umbral alterado puede suprimir una alerta real. JWT evita mantener estado de sesión en el servidor.
Pruebas	JUnit + Postman	Según Spring Boot (JUnit)	Pruebas manuales	AC-02 y AC-05: verificación repetible de la evaluación en ≤ 5 s y de la entrega por 2 canales, según los escenarios de la sección 1.3.
Control de versiones	Git + GitHub	—	Ninguna	Restricción del curso: el DAS, los ADRs y el código se versionan en el repositorio del proyecto.
Entrada de SMS	Endpoint HTTP (webhook) que recibe el SMS; en desarrollo se simula con Postman	—	Contratar un gateway SMS de pago en esta etapa	RT-02 y RA-02: el canal de los sensores sigue siendo SMS; el mensaje llega al sistema como una petición HTTP, sin costo de un gateway de pago.
Canales de salida	Correo (SMTP) + Telegram Bot	—	SMS saliente por gateway de pago	AC-05 (Confiabilidad): dos canales independientes, ambos sin costo (RA-02), de modo que la falla de uno no impide la entrega por el otro.

Nota: las versiones marcadas como "Según Spring Boot" se heredan de su gestión de dependencias; todas las versiones se confirman al crear el proyecto.

Sección 3: Vistas de Arquitectura (Modelo C4 y UML)

Esta sección presenta la arquitectura mediante el Modelo C4 con notación UML estándar. Cada vista incluye su diagrama, una leyenda y un párrafo de lectura. Las fuentes de los diagramas (.puml) y sus exportaciones (.png) están en la carpeta docs/diagramas/ del repositorio.

3.1 Vista de Contexto — C4 Nivel 1

Muestra el sistema como una caja negra: quién lo usa y con qué dispositivos y sistemas externos se comunica, sin mostrar ninguna tecnología interna.

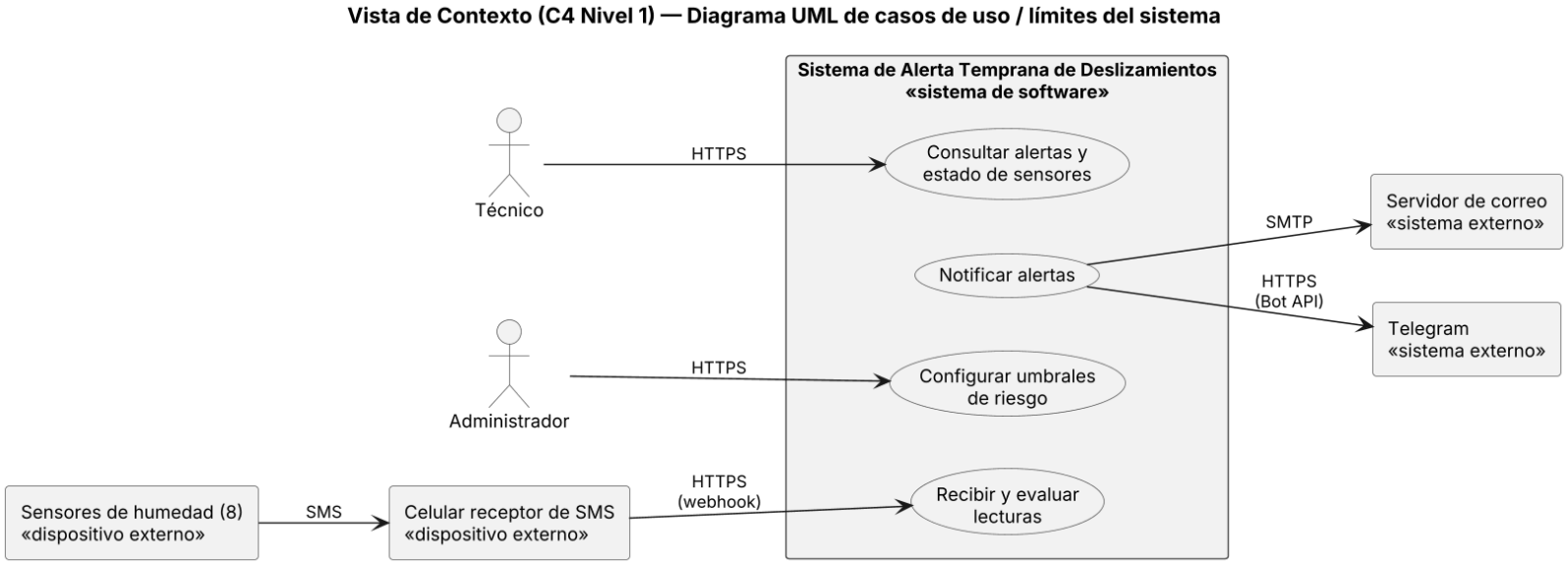


Diagrama UML de la Vista de Contexto (C4 Nivel 1)

Leyenda

Elemento del diagrama	Qué representa
Figura de persona	Actor humano que usa el sistema.
Rectángulo «dispositivo externo»	Equipo físico fuera del sistema que no se modifica (los sensores y el celular receptor de SMS).
Rectángulo «sistema externo»	Software de terceros con el que el sistema se comunica (servidor de correo y Telegram).
Recuadro grande con elipses	Límite del sistema y sus capacidades principales.
Flecha con etiqueta	Apunta hacia quien recibe la comunicación; la etiqueta indica el protocolo.

Actores y relación con las historias de usuario (sección 1.2): el administrador (HU-02) y el técnico (HU-03) son los actores humanos y usan el sistema por HTTPS. La HU-01 la ejecuta el propio sistema al recibir y evaluar las lecturas, por eso no aparece como un actor externo. Cada interfaz externa figura con su protocolo: SMS desde los sensores, HTTPS (webhook) desde el celular receptor, SMTP hacia el servidor de correo y HTTPS (Bot API) hacia Telegram.

Lectura en 30 segundos: ocho sensores de humedad envían sus lecturas por SMS a un celular que ya existe en el caso. Ese celular las reenvía por HTTPS (webhook) al sistema, que las evalúa, genera alertas y las notifica por correo (SMTP) y por Telegram (HTTPS). El técnico consulta las alertas y el estado de los sensores, y el administrador configura los umbrales de riesgo, ambos por HTTPS. Los sensores y el celular no se modifican: todo el valor sale del software.

Nota: el reenvío del SMS desde el celular receptor hacia el webhook es un supuesto de diseño por confirmar con el cliente. El caso establece que los sensores envían sus lecturas por SMS a un celular y que su hardware no puede modificarse (RT-01), pero no indica cómo llega ese mensaje a un sistema de software. El mecanismo de reenvío aún no está definido y deberá ser compatible con la restricción de presupuesto (RA-02), es decir, sin un gateway SMS de pago; mientras tanto, en desarrollo el SMS se simula con Postman (sección 2.2).

3.2 Vista de Contenedores — C4 Nivel 2

Abre la caja negra del sistema: muestra las piezas que se ejecutan por separado y los almacenes de datos, cada una con su tecnología entre corchetes.

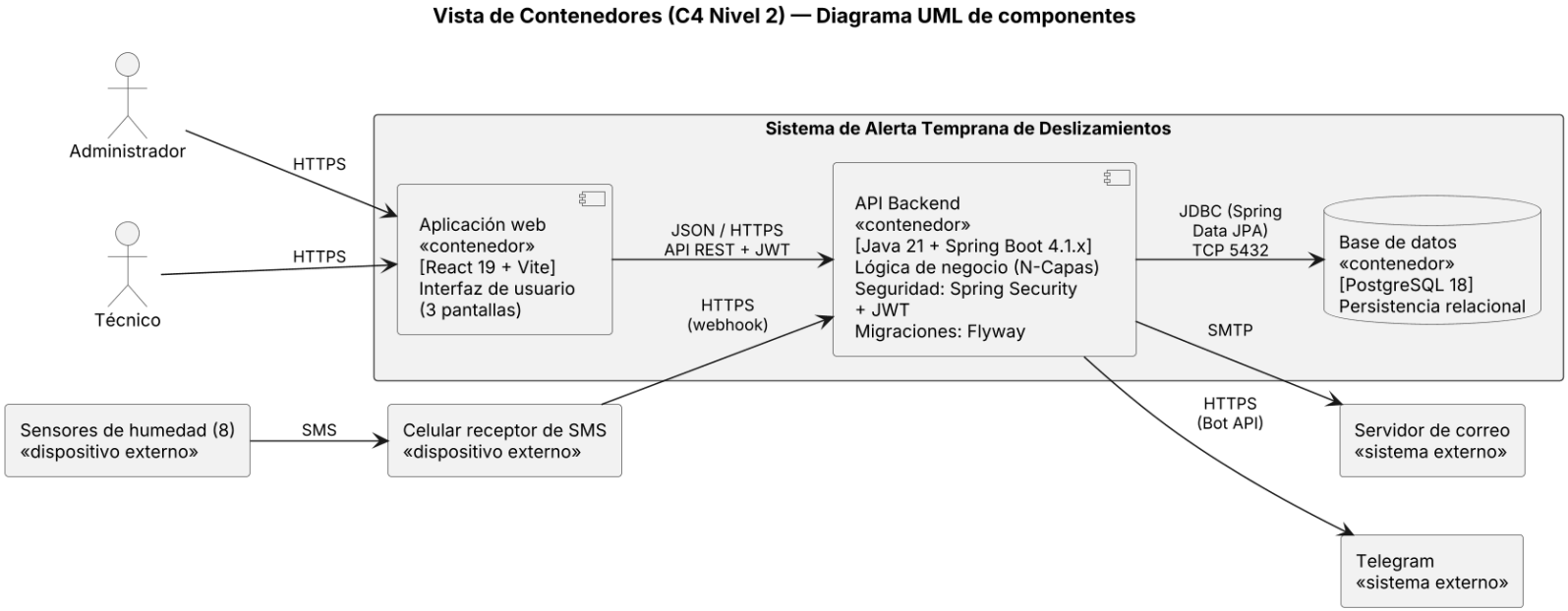


Diagrama UML de componentes de la Vista de Contenedores (C4 Nivel 2)

Leyenda

Elemento del diagrama	Qué representa
Componente «contenedor» con la tecnología entre corchetes	Pieza propia que se ejecuta por separado: la aplicación web y la API.
Cilindro «contenedor»	Almacén de datos propio: la base de datos.
Recuadro grande	Límite del sistema: lo de adentro es propio, lo de afuera no se controla.
Rectángulos «dispositivo externo» y «sistema externo»	Los mismos elementos externos de la Vista de Contexto.
Flecha con etiqueta	Apunta hacia quien recibe la comunicación; la etiqueta indica el protocolo (y el puerto cuando aplica).

Correspondencia con la pila tecnológica (sección 2.2): la aplicación web es React 19 con Vite; la API es Java 21 con Spring Boot 4.1.x, que incluye Spring Security con JWT, Spring Data JPA y Flyway; y la base de datos es PostgreSQL 18. El webhook es la entrada de los SMS, y el correo (SMTP) y Telegram son los canales de salida. Las tres capas del patrón N-Capas (sección 2.1) están dentro del contenedor API Backend.

Lectura en 30 segundos: el técnico y el administrador entran por HTTPS a la aplicación web (React), que habla con la API (Spring Boot) en JSON por HTTPS con JWT. La API es la única pieza que accede a la base de datos PostgreSQL (JDBC, puerto 5432). Además recibe por webhook HTTPS los SMS que reenvía el celular receptor y envía las alertas por correo (SMTP) y por Telegram (HTTPS). Las tres piezas dentro del recuadro son propias; los dispositivos y sistemas de afuera no se controlan.